



Working with Gen AI APIs

Hany Saad
hghiett@iti.gov.eg



Course: Developing Gen-AI based software



Course Title: AI-powered Coding Assistant tools

Tracks: Development tracks only (Backend development tracks only)

Duration: 18 hours (3 Lectures + 3 Labs)

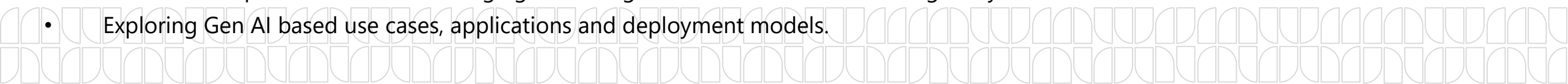
Evaluation: Attendance + Labs.

Prerequisites:

- Intro to Generative AI and prompt engineering.
- REST API (To be after Web API course in each dev track)

Outlines:

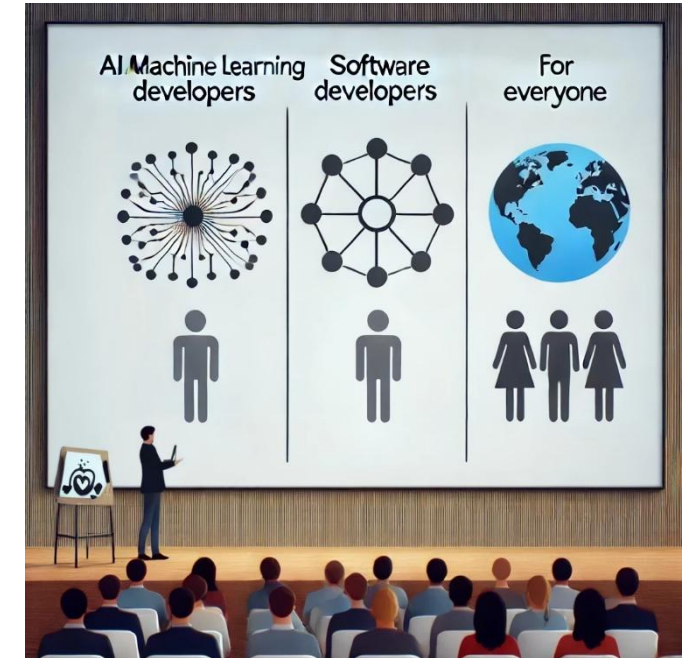
- Gen AI Providers, models and capabilities (OpenAI, Google, Azure OpenAI Service, and Anthropic models): Text, Vision, Image, TTS, STT models.
- Working with OpenAI APIs: Text completions, Image generation and vision APIs (OpenAI APIs or open-source models APIs)
- Types of Prompts: System, assistant, and user prompts, and how to write better prompts for better results.
- Advanced API features: Structured output, Assistant API, Function calling.
- Foundation models and models fine-tuning.
- Open-source models and models catalog platforms: Hugging-face, MS Azure AI, and GitHub models.
- Intro to RAG and AI orchestration platforms: Langchain and MS Semantic Kernel.
- Intro to Agentic AI frameworks and tools for AI Agents development.
- Gen AI development trends and emerging technologies: VLMs, LAMs, and multi-agent systems.
- Exploring Gen AI based use cases, applications and deployment models.



Why do we need to know about Gen AI?

Who should learn about Gen AI?

- **AI / Machine Learning Developers:**
 - Design and refine Generative AI models to create more intelligent and efficient AI-driven solutions.
- **Software Developers:**
 - Utilize AI tools to enhance productivity in development tasks.
 - Leverage Generative AI APIs to build AI-powered applications
- **Everyone:**
 - In today's AI-centric world, it's crucial for individuals across all professions to grasp the basics of Generative AI and incorporate AI tools into their workflows to boost productivity.



Gen-AI based startups and real products



- **Use cases for real products using Gen AI:**
 - Loop-x (AI Agents for different roles): <https://loop-x.co>
 - Gemelo ai (Twin AI creator): <https://gemelo.ai>
 - Apriora AI (Recruitment and Interviews): <https://www.apriora.ai>
 - Velents AI (Recruitment and Interviews): : <https://www.velents.com>
 - Nancy AI (Recruitment and Interviews): : <https://nancy-ai.com>
 - Mariana AI (Medical): <https://marianaai.com>



How to use the AI tools responsibly?

- **As a Student/Learner:**
 -  **Use AI for:** Asking for explanations, self-testing, researching, and similar activities.
 -  **Don't use AI for:** Assignments, projects, tasks, or topics you're still learning.
 - **Build your skills** to work effectively both with and without AI.
- **In a Job or Freelancing:**
 - **Check first** if AI tool usage is permitted and which tools are allowed.
 - **Use AI to boost productivity** and improve your work.
 - **Always review** AI-generated content carefully.
 - **Don't rely on AI** for tasks you have no prior knowledge of.

Models and Capabilities



OpenAI Models

<https://platform.openai.com/docs/models>

Models overview

The OpenAI API is powered by a diverse set of models with different capabilities and price points. You can also make customizations to our models for your specific use case with [fine-tuning](#).

MODEL	DESCRIPTION
GPT-4o	Our high-intelligence flagship model for complex, multi-step tasks
GPT-4o mini	Our affordable and intelligent small model for fast, lightweight tasks
o1-preview and o1-mini	Language models trained with reinforcement learning to perform complex reasoning.
GPT-4 Turbo and GPT-4	The previous set of high-intelligence models
GPT-3.5 Turbo	A fast, inexpensive model for simple tasks
DALL·E	A model that can generate and edit images given a natural language prompt
TTS	A set of models that can convert text into natural sounding spoken audio
Whisper	A model that can convert audio into text
Embeddings	A set of models that can convert text into a numerical form
Moderation	A fine-tuned model that can detect whether text may be sensitive or unsafe

DeepSeek AI Models

https://api-docs.deepseek.com/quick_start/pricing

Pricing Details

MODEL ⁽¹⁾		deepseek-chat	deepseek-reasoner
CONTEXT LENGTH		64K	64K
MAX COT TOKENS ⁽²⁾		-	32K
MAX OUTPUT TOKENS ⁽³⁾		8K	8K
STANDARD PRICE (UTC 00:30-16:30)	1M TOKENS INPUT (CACHE HIT) ⁽⁴⁾	\$0.07	\$0.14
	1M TOKENS INPUT (CACHE MISS)	\$0.27	\$0.55
	1M TOKENS OUTPUT ⁽⁵⁾	\$1.10	\$2.19
DISCOUNT PRICE ⁽⁶⁾ (UTC 16:30-00:30)	1M TOKENS INPUT (CACHE HIT)	\$0.035 (50% OFF)	\$0.035 (75% OFF)
	1M TOKENS INPUT (CACHE MISS)	\$0.135 (50% OFF)	\$0.135 (75% OFF)
	1M TOKENS OUTPUT	\$0.550 (50% OFF)	\$0.550 (75% OFF)

- (1) The `deepseek-chat` model points to **DeepSeek-V3**. The `deepseek-reasoner` model points to **DeepSeek-R1**.
- (2) **CoT (Chain of Thought)** is the reasoning content `deepseek-reasoner` gives before output the final answer. For details, please refer to [Reasoning Model](#).

Google AI Models

Model name	Modality	Description
Gemini 1.5 Flash	Language, audio, vision	The fastest, most cost-effective Gemini multimodal model. It's built for high volume tasks and latency-sensitive, affordable applications. Because of how responsive Gemini 1.5 Flash is, it's a good option to create chat assistants and on-demand content generation applications.
Gemini 1.5 Pro	Language, audio, vision	Multimodal model that supports adding image, audio, video, and PDF files in text or chat prompts for a text or code response.
Gemini 1.0 Pro	Language	Designed to handle natural language tasks, multiturn text and code chat, and code generation.
Gemini 1.0 Pro Vision	Language, vision	Multimodal model that supports adding image, video, and PDF files in text or chat prompts for a text or code response.
PaLM 2 for Text	Language	Fine-tuned to follow natural language instructions and is suitable for a variety of language tasks.
PaLM 2 for Chat	Language	Fine-tuned to conduct natural conversation. Use this model to build and customize your own chatbot application.
Codey for Code Completion	Language	Generates code based on code prompts. Good for code suggestions and minimizing bugs in code.
Codey for Code Generation	Language	Generates code based on natural language input. Good for writing functions, classes, unit tests, and more.
Codey for Code Chat	Language	Get code-related assistance through natural conversation. Good for questions about an API, syntax in a supported language, and more.

Embeddings for Text	Language	Converts textual data into numerical vectors that can be processed by machine learning algorithms, especially large models.
Imagen for Image Generation	Vision	Create or edit studio-grade images at scale using text prompts.
Vertex Image Segmentation (Preview)	Vision	Use text prompts or draw scribbles to segment an image. Image segmentation lets you, for example, detect objects, remove the background of an image, or segment the foreground of an image.
Imagen for Captioning & VQA	Language	Generates a relevant description for a given image.
Embeddings for Multimodal	Vision	Generates vectors based on images, which can be used for downstream tasks like image classification and image search.
Chirp	Speech	A version of a Universal Speech Model that has over 2B parameters and can transcribe in over 100 languages in a single model.

<https://cloud.google.com/vertex-ai/generative-ai/docs/model-garden/explore-models>

Anthropic

AI Models

<https://docs.anthropic.com/en/docs/about-claude/models>

	Claude 3.5 Sonnet	Claude 3 Opus	Claude 3 Sonnet	Claude 3 Haiku
Description	Most intelligent model	Powerful model for highly complex tasks	Balance of intelligence and speed	Fastest and most compact model for near-instant responsiveness
Strengths	Highest level of intelligence and capability	Top-level performance, intelligence, fluency, and understanding	Strong utility, balanced for scaled deployments	Quick and accurate targeted performance
Multilingual	Yes	Yes	Yes	Yes
Vision	Yes	Yes	Yes	Yes
Message Batches API	Yes	Yes	No	Yes
API model name	Upgraded version: claude-3-5-sonnet-20241022	claude-3-opus-20240229	claude-3-sonnet-20240229	claude-3-haiku-20240307

Qwen AI Models

<https://www.alibabacloud.com/help/en/model-studio/getting-started/models>

Flagship models

Flagship models	 Qwen-Max Best suited for complex tasks with the highest capability	 Qwen-Plus Balanced performance, speed, and cost	 Qwen-Turbo Best suited for simple tasks with high speed and low cost
Maximum context length (tokens)	32,768	131,072	1,008,192
Minimum input price (per 1,000 tokens)	\$0.0016	\$0.0004	\$0.00005
Minimum output price (per 1,000 tokens)	\$0.0064	\$0.0012	\$0.0002

Model overview

Category	Model	Description
Text generation	Qwen	- Qwen large language model: Commercial version - Multi-modal model: Visual understanding model C - Translation model: Qwen translation model
Vector	Text embedding	Converts text into a set of numerical representations :

MS Azure OpenAI Service

<https://azure.microsoft.com/en-us/products/ai-services/openai-service>

Models	Description
o1-preview and o1-mini	Limited access models, specifically designed to tackle reasoning and problem-solving tasks with increased focus and capability.
GPT-4o & GPT-4o mini & GPT-4 Turbo	The latest most capable Azure OpenAI models with multimodal versions, which can accept both text and images as input.
GPT-4o audio	A GPT-4o model that supports low-latency, "speech in, speech out" conversational interactions.
GPT-4	A set of models that improve on GPT-3.5 and can understand and generate natural language and code.
GPT-3.5	A set of models that improve on GPT-3 and can understand and generate natural language and code.
Embeddings	A set of models that can convert text into numerical vector form to facilitate text similarity.
DALL-E	A series of models that can generate original images from natural language.
Whisper	A series of models in preview that can transcribe and translate speech to text.
Text to speech (Preview)	A series of models in preview that can synthesize text to speech.

Open Source models



Hugging face platform

<https://huggingface.co/models>

users...

 Models  Datasets  Spaces

Models 1,069,708

 Filter by name

 nvidia/Llama-3.1-Nemotron-70B-Instruct-HF

 Text Generation • Updated 8 days ago •  89.1k •  1.1k

 stabilityai/stable-diffusion-3.5-large

 Text-to-Image • Updated 2 days ago •  11k •  •  481

 genmo/mochi-1-preview

 Text-to-Video • Updated about 15 hours ago •  413

 deepseek-ai/Janus-1.3B

 Any-to-Any • Updated 4 days ago •  4.64k •  369

 nvidia/Llama-3.1-Nemotron-70B-Instruct

Updated 7 days ago •  16.7k •  397

MS Azure AI Models

<https://ai.azure.com/explore/models>

Find the right model to build your custom AI solution

Announcements

Phi-3 is now available

Microsoft's Phi-3-mini SLMs offer groundbreaking performance at a small size

[View models](#) [Read blog](#)

Build the future of AI with Meta Llama 3

Serverless APIs for Meta-Llama-3-8B-Instruct and Meta-Llama-3-70B-Instruct models

[View models](#) [Read blog](#)

Cohere models are here!

Cohere's Enterprise AI models include Command R, Command R+,

[View models](#) [Read blog](#)

All filters Collections Inference tasks Fine-tuning tasks Licenses

Search

 gpt-4  Chat completion	 gpt-35-turbo-instruct  Chat completion	 davinci-002  Completions	 text-embedding-ada-002  Embeddings	 gpt-4-32k  Chat completion	 gpt-35  Chat completion
 gpt-35-turbo  Chat completion	 babbage-002  Completions	 mistralai-Mistral-8x7B-Instruct-v0.1  Chat completion	 mistralai-Mistral-8x22B-v0.1  Text generation	 mistralai-Mistral-8x22B-Instruct-v0.1  Chat completion	 mistralai-Mistral-8x22B-Instruct-v0.1  Text generation
 mistralai-Mistral-8x7B-v0.1  Text generation	 mistralai-Mistral-7B-Instruct-v0.1  Chat completion	 mistralai-Mistral-7B-v0.1  Text generation	 mistralai-Mistral-7B-Instruct-v0.1  Chat completion	 Mistral-large  Chat completion	 Meta-Llama-3-8B-Instruct  Chat completion
 Meta-Llama-3-8B-Instruct  Chat completion	 Meta-Llama-3-8B  Text generation	 Meta-Llama-3-70B  Text generation	 Llama-2-70b  Text generation	 Llama-2-7b  Text generation	 Llama-2-70b  Text generation
 CodeLlama-7b-hf  Text generation	 CodeLlama-7b-Python-hf  Text generation	 CodeLlama-7b-Instruct-hf  Text generation	 CodeLlama-34b-hf  Text generation	 CodeLlama-34b-Python-hf  Text generation	 CodeLlama-34b-Python-hf  Text generation
 CodeLlama-13b-hf  Text generation	 CodeLlama-13b-Python-hf  Text generation	 CodeLlama-13b-Instruct-hf  Text generation	 Llama-2-7b-chat  Chat completion	 Llama-2-70b-chat  Chat completion	 Llama-2-70b-chat  Chat completion
 Nemotron-3-8B-Chat-SteerLM  Text generation	 Nemotron-3-8B-Chat-RLHF  Text generation	 Nemotron-3-8B-Chat-SFT  Text generation	 Nemotron-3-8B-QA-4k  Text generation	 Nemotron-3-8B-Base-4k  Text generation	 Phi-3-mini-128k-instruct  Chat completion
 Phi-3-mini-128k-instruct  Chat completion	 Cohere-embed-v3-multilingual  Embeddings	 Cohere-embed-v3-english  Embeddings	 Cohere-command-r-plus  Chat completion	 Cohere-command-r  Chat completion	 Deci-Decoder-7B  Text generation
 Deci-Decoder-7B  Text generation	 Deci-Decoder-1b  Text generation	 tiiuae-falcon-7b-instruct  Text generation	 tiiuae-falcon-7b  Text generation	 tiiuae-falcon-40b  Text generation	 tiiuae-falcon-40b  Text generation

Prev Next

GitHub Models

<https://github.com/marketplace/models>

Enhance your workflow with extensions

Tools from the community and partners to simplify tasks and automate processes

Search for Copilot extensions, apps, actions, and models



Featured

Copilot

Models

Apps

Actions

Create a new extension

Models

Try, test, and deploy from a wide range of model types, sizes, and specializations. [Learn more.](#)

Beta Give feedback

 AI21-Jamba-Instruct Model Jamba-Instruct is the world's first production-grade Mamba-based LLM model and leverages its hybrid Mamba-Transformer architecture to...	 Cohere Command R Model Command R is a scalable generative model targeting RAG and Tool Use to enable production-scale AI for enterprise.
 Cohere Command R+ Model Command R+ is a state-of-the-art RAG-optimized model designed to tackle enterprise-grade workloads.	 Cohere Embed v3 English Model Cohere Embed English is the market's leading text representation model used for semantic search, retrieval-augmented generation (RA...
 Cohere Embed v3 Multilingual Model Cohere Embed Multilingual is the market's leading text representation model used for semantic search, retrieval-augmented generation (RA...	 Meta-Llama-3-70B-Instruct Model A powerful 70-billion parameter model excelling in reasoning, coding, and broad language applications.
 Meta-Llama-3-8B-Instruct Model A versatile 8-billion parameter model optimized for dialogue and text generation tasks.	 Meta-Llama-3.1-405B-Instruct Model The Llama 3.1 instruction tuned text only models are optimized for multilingual dialogue use cases and outperform many of the available...
 Meta-Llama-3.1-70B-Instruct Model The Llama 3.1 instruction tuned text only models are optimized for multilingual dialogue use cases and outperform many of the available...	 Meta-Llama-3.1-8B-Instruct Model The Llama 3.1 instruction tuned text only models are optimized for multilingual dialogue use cases and outperform many of the available...
 Mistral Large Model Mistral's flagship model that's ideal for complex tasks that require large reasoning capabilities or are highly specialized (Synthetic Text...	 Mistral Large (2407) Model Mistral Large (2407) is an advanced Large Language Model (LLM) with state-of-the-art reasoning, knowledge and coding capabilities.
 Mistral Nemo Model Mistral Nemo is a cutting-edge Language Model (LLM) boasting state-of-the-art reasoning, world knowledge, and coding capabilities within...	 Mistral Small Model Mistral Small can be used on any language-based task that requires high efficiency and low latency.
 OpenAI GPT-4o Model OpenAI's most advanced multimodal model in the GPT-4 family. Can handle both text and image inputs.	 OpenAI GPT-4o mini Model An affordable, efficient AI solution for diverse text and image tasks.
 OpenAI Text Embedding 3 (large) Model Text-embedding-3 series models are the latest and most capable embedding model from OpenAI.	 OpenAI Text Embedding 3 (small) Model Text-embedding-3 series models are the latest and most capable embedding model from OpenAI.
 Phi-3 medium instruct (128k) Model Same Phi-3-medium model, but with a larger context size for RAG or few shot prompting.	 Phi-3 medium instruct (4k) Model A 14B parameters model, proves better quality than Phi-3-mini, with a focus on high-quality, reasoning-dense data.
 Phi-3 mini instruct (128k) Model Same Phi-3-mini model, but with a larger context size for RAG or few shot prompting.	 Phi-3 mini instruct (4k) Model Tiniest member of the Phi-3 family. Optimized for both quality and low latency.
 Phi-3 small instruct (128k) Model Same Phi-3-small model, but with a larger context size for RAG or few shot prompting.	 Phi-3 small instruct (8k) Model A 7B parameters model, proves better quality than Phi-3-mini, with a focus on high-quality, reasoning-dense data.

Getting started with OpenAI APIs



Getting started with OpenAI APIs – Data Privacy

As of March 1, 2023, data sent to the OpenAI API will not be used to train or improve OpenAI models (unless you explicitly opt-in to share data with us, such as by [providing feedback in the Playground](#)). One advantage to opting in is that the models may get better at your use case over time.

To help identify abuse, API data may be retained for up to 30 days, after which it will be deleted (unless otherwise required by law). For trusted customers with sensitive applications, zero data retention may be available. With zero data retention, request and response bodies are not persisted to any logging mechanism and exist only in memory in order to serve the request.

Note that this data policy does not apply to OpenAI's non-API consumer services like [ChatGPT](#) or [DALL·E Labs](#).

Getting started with OpenAI APIs – Important terms

Text completion: is a specific task that can be performed by GPT models. Given a partial sentence or a prompt, the model can generate a plausible and coherent continuation of the text. This text completion ability allows GPT models to assist with tasks such as writing assistance, content generation, answering questions, and more.

Chat completion is a specific type of text completion that is designed to simulate a conversation or dialogue between two or more participants. While text completion focuses on generating coherent and contextually relevant text based on a given prompt, chat completion goes a step further by incorporating the conversational aspect.

Understanding Tokens:

Tokens can be thought of as pieces of words. Before the API processes the prompts, the input is broken down into tokens. These tokens are not cut up exactly where the words start or end - tokens can include trailing spaces and even sub-words. Here are some helpful rules of thumb for understanding tokens in terms of lengths:

- 1 token $\sim$ 4 chars in English

- 1 token $\sim$ $\frac{3}{4}$ words

- 100 tokens $\sim$ 75 words

- Or

- 1-2 sentence $\sim$ 30 tokens

- 1 paragraph $\sim$ 100 tokens

- 1,500 words $\sim$ 2048 tokens

Getting started with OpenAI APIs - Auth

Getting started:

- First, you need to create an OpenAI account.
- Go to [OpenAI platform](#)
- Create API_Key from [here](#)
- Finally, you can access the APIs as REST APIs from [here](#), or you can use provided libraries for different languages [here](#)

Documentation and pricing:

- OpenAI API models [pricing](#).
- OpenAI APIs [Docs](#).
- API [Reference](#)
- OpenAI [Examples](#).
- OpenAI [Cookbook](#)

How to work with APIs

- Using Language specific SDK
- Using REST API

Getting started with OpenAI APIs – Making Requests

Making requests

You can paste the command below into your terminal to run your first API request. Make sure to replace

`$OPENAI_API_KEY` with your secret API key. If you are using a legacy user key and you have multiple projects, you will also need to **specify the Project Id**. For improved security, we recommend transitioning to project based keys instead.

```
1 curl https://api.openai.com/v1/chat/completions \  
2   -H "Content-Type: application/json" \  
3   -H "Authorization: Bearer $OPENAI_API_KEY" \  
4   -d '{  
5     "model": "gpt-4o-mini",  
6     "messages": [{"role": "user", "content": "Say this is a test!"}],  
7     "temperature": 0.7  
8   }'
```

<https://platform.openai.com/docs/api-reference/chat/create>

Getting started with OpenAI APIs – Request parameters

Request body

messages array **Required**

A list of messages comprising the conversation so far. Depending on the **model** you use, different message types (modalities) are supported, like **text**, **images**, and **audio**.

^ Hide possible types

System message	object
▼ Show properties	
User message	object
▼ Show properties	
Assistant message	object
▼ Show properties	

temperature number or null Optional Defaults to 1

What sampling temperature to use, between 0 and 2. Higher values like 0.8 will make the output more random, while lower values like 0.2 will make it more focused and deterministic.

We generally recommend altering this or `top_p` but not both.

response_format object Optional

An object specifying the format that the model must output. Compatible with **GPT-4o**, **GPT-4o mini**, **GPT-4 Turbo** and all GPT-3.5 Turbo models newer than `gpt-3.5-turbo-1106`.

Setting to `{ "type": "json_schema", "json_schema": {...} }` enables Structured Outputs which ensures the model will match your supplied JSON schema. Learn more in the **Structured Outputs guide**.

max_completion_tokens integer or null Optional

An upper bound for the number of tokens that can be generated for a completion, including visible output tokens and **reasoning tokens**.

n integer or null Optional Defaults to 1

How many chat completion choices to generate for each input message. Note that you will be charged based on the number of generated tokens across all of the choices. Keep `n` as `1` to minimize costs.

n integer or null Optional Defaults to 1

How many chat completion choices to generate for each input message. Note that you will be charged based on the number of generated tokens across all of the choices. Keep `n` as `1` to minimize costs.

Getting started with OpenAI APIs – Message roles

```
1  # Note: you need to be using OpenAI Python v0.27.0 for the code below to
2  import openai
3
4  openai.ChatCompletion.create(
5      model="gpt-3.5-turbo",
6      messages=[
7          {"role": "system", "content": "You are a helpful assistant."},
8          {"role": "user", "content": "Who won the world series in 2020?"},
9          {"role": "assistant", "content": "The Los Angeles Dodgers won the World"},
10         {"role": "user", "content": "Where was it played?"}
11     ]
12 )
```



OpenAI APIs – Image generation (API Reference)

POST <https://api.openai.com/v1/images/generations>

Creates an image given a prompt.

Request body

prompt string **Required**

A text description of the desired image(s). The maximum length is 1000 characters for `dall-e-2` and 4000 characters for `dall-e-3`.

model string Optional Defaults to `dall-e-2`

The model to use for image generation.

n integer or null Optional Defaults to 1

The number of images to generate. Must be between 1 and 10. For `dall-e-3`, only `n=1` is supported.

quality string Optional Defaults to `standard`

The quality of the image that will be generated. `hd` creates images with finer details

Example request

curl ▾



```
1 curl https://api.openai.com/v1/images/generation
2   -H "Content-Type: application/json" \
3   -H "Authorization: Bearer $OPENAI_API_KEY" \
4   -d '{
5     "model": "dall-e-3",
6     "prompt": "A cute baby sea otter",
7     "n": 1,
8     "size": "1024x1024"
9   }'
```

Response



```
1 {
2   "created": 1589478378,
3   "data": [
4     {
5       "url": "https://..."
6     },
7   ]
8 }
```

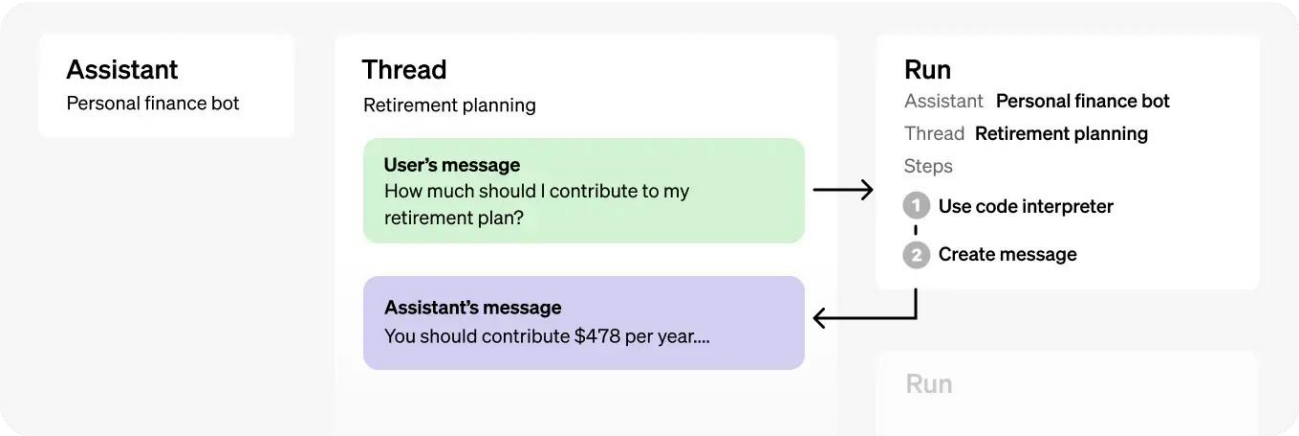
OpenAI APIs – Assistants API Overview

Assistants API Overview

The Assistants API allows you to build AI assistants within your own applications. An Assistant has instructions and can leverage models, tools, and files to respond to user queries. The Assistants API currently supports three types of [tools](#): Code Interpreter, File Search, and Function calling.

How Assistants work

- 1.Assistants can call OpenAI’s [models](#) with specific instructions to tune their personality and capabilities.
- 2.Assistants can access **multiple tools in parallel**. These can be both OpenAI-hosted tools — like [code interpreter](#) and [file search](#) — or tools you build / host (via [function calling](#)).
- 3.Assistants can access **persistent Threads**. Threads simplify AI application development by storing message history and truncating it when the conversation gets too long for the model’s context length. You create a Thread once, and simply append Messages to it as your users reply.
- 4.Assistants can access files in several formats — either as part of their creation or as part of Threads between Assistants and users. When using tools, Assistants can also create files (e.g., images, spreadsheets, etc) and cite files they reference in the Messages they create.



OBJECT	WHAT IT REPRESENTS
Assistant	Purpose-built AI that uses OpenAI's models and calls tools
Thread	A conversation session between an Assistant and a user. Threads store Messages and automatically handle truncation to fit content into a model's context.
Message	A message created by an Assistant or a user. Messages can include text, images, and other files. Messages stored as a list on the Thread.
Run	An invocation of an Assistant on a Thread. The Assistant uses its configuration and the Thread's Messages to perform tasks by calling models and tools. As part of a Run, the Assistant appends Messages to the Thread.
Run Step	A detailed list of steps the Assistant took as part of a Run. An Assistant can call tools or create Messages during its run. Examining Run Steps allows you to introspect how the Assistant is getting to its final results.

Assistant API – Code interpreter

Code Interpreter

What it is: The Code Interpreter allows an OpenAI Assistant to write and run Python code within a secure environment.

Use Case: Suppose you need to analyze a CSV file containing sales data. The Code Interpreter can read the file, perform calculations (like summing up total sales), and even create a graph to visualize the data. If the code doesn't work initially, it can try different approaches until it gets it right.

Example: You ask, "Analyze this sales data and show me the total revenue." The Assistant can load your CSV file, write the necessary Python code, and present you with the result.

Assistant API – File search

File Search

What it is: File Search lets the Assistant access and retrieve information from documents you upload, beyond its pre-existing knowledge.

Use Case: Imagine you upload a manual for a product your company makes. The Assistant can search through that manual to answer specific questions, like "What's the warranty period?" by finding the relevant section in the document.

Example: You upload a PDF of a product guide and ask, "What are the installation steps?" The Assistant can locate and provide the steps directly from your document.

Assistant API – Function calling

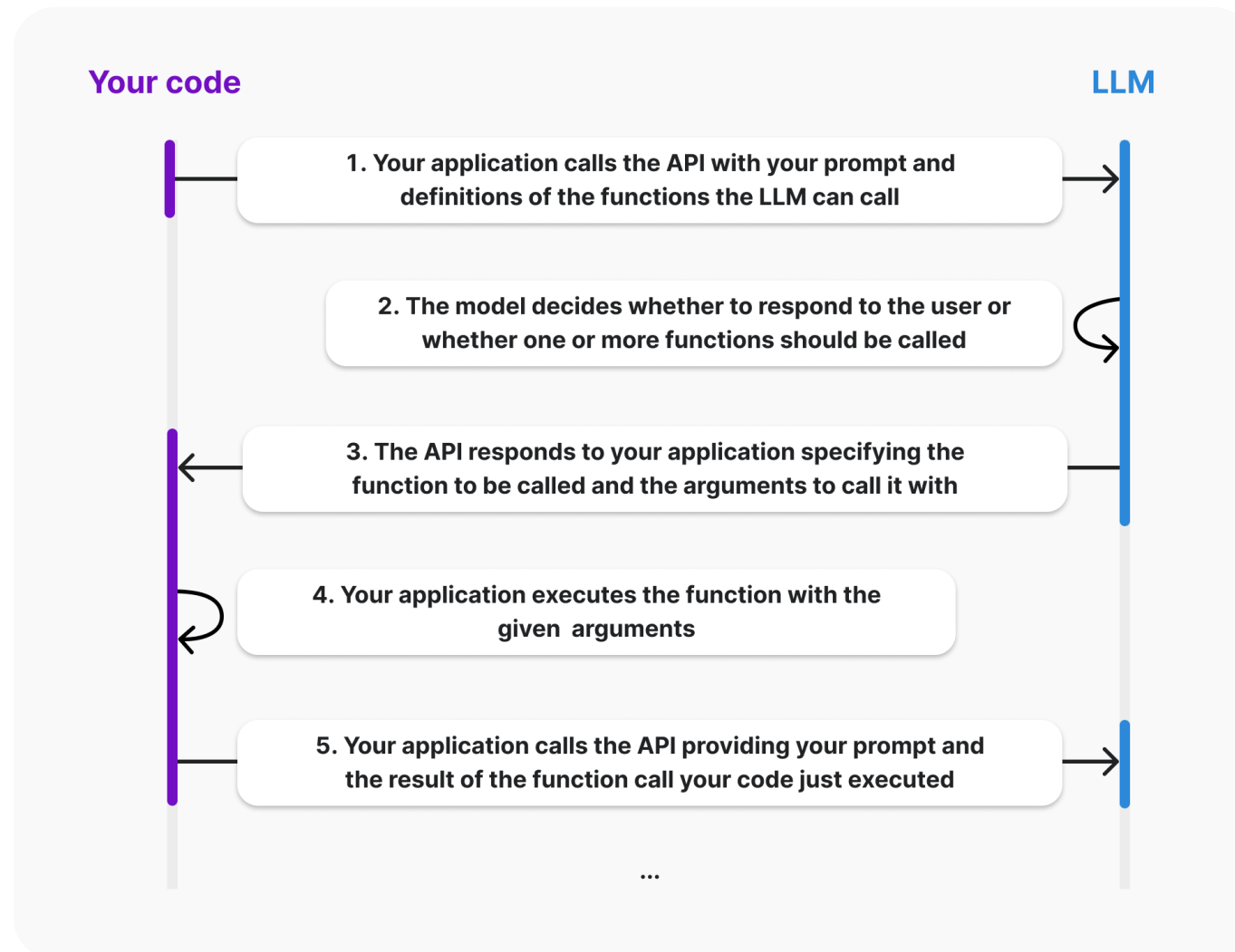
Function Calling

What it is: Function Calling allows you to define specific functions that the Assistant can call during a conversation. The Assistant knows when to use these functions and what arguments to pass to them.

Use Case: If you have a weather forecasting function, the Assistant can automatically call this function when you ask about the weather, providing you with accurate, up-to-date information.

Example: You might ask, "What's the weather like today?" The Assistant will recognize this as a request for weather data and use the appropriate function to fetch and return the current weather for your location.

Assistant API – Function calling (Cont.)



Structured output

What it is: Structured Outputs ensure that the responses generated by the OpenAI Assistant are always formatted according to a specified JSON Schema.

Use Case: Suppose you're building an app that needs to process user input, such as booking information. You define a JSON Schema that specifies exactly how the booking details should be structured (e.g., with fields like: name, date, time). The Assistant will always provide the output in this structured format, reducing the risk of errors.

Example: If you ask the Assistant to "Create a booking for John on September 10th at 3 PM," it will return the details in the exact JSON format you specified, like this:

```
{ "name": "John", "date": "2024-09-10", "time": "15:00"}
```

This ensures the data is always ready to be processed by your application without additional checks.

```
# Your OpenAI API key
openai.api_key = "your-api-key"

# Define your JSON Schema
json_schema = {
    "type": "object",
    "properties": {
        "name": {"type": "string"},
        "date": {"type": "string", "format": "date"},
        "time": {"type": "string", "format": "time"}
    },
    "required": ["name", "date", "time"]
}

# Make a request with the schema
response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Create a booking for John on September 10th at 3 PM."}
    ],
)
```

Vision

Introduction

GPT-4o, GPT-4o mini, and GPT-4 Turbo have vision capabilities, meaning the models can take in images and answer questions about them. Historically, language model systems have been limited by taking in a single input modality, text.

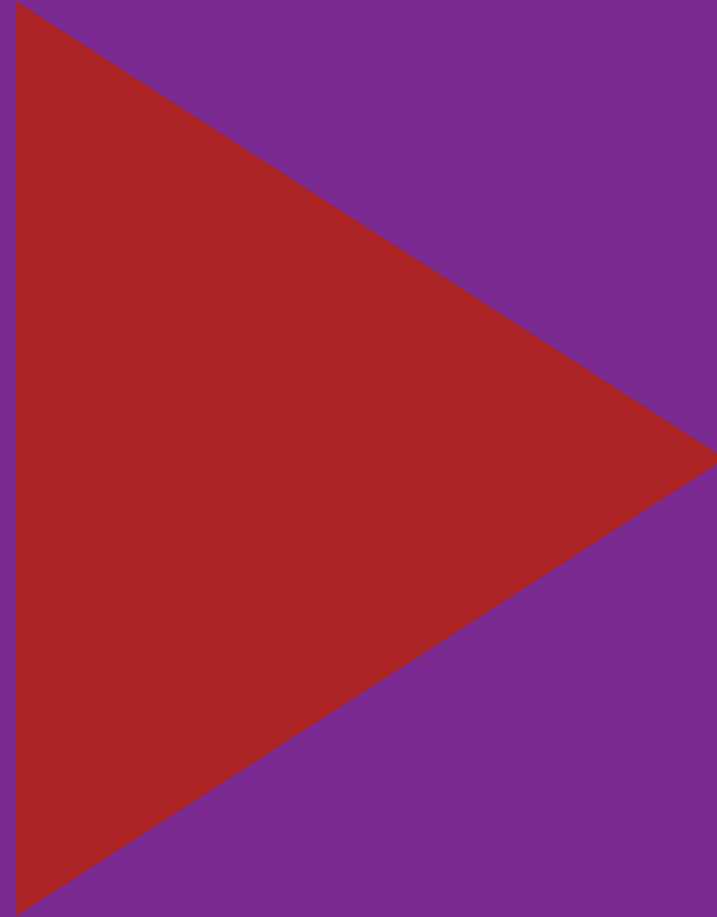
```
5 response = client.chat.completions.create(  
6     model="gpt-4o-mini",  
7     messages=[  
8         {  
9             "role": "user",  
10            "content": [  
11                {"type": "text", "text": "What's in this image?"},  
12                {  
13                    "type": "image_url",  
14                    "image_url": {  
15                        "url": "https://upload.wikimedia.org/wikipedia/commons/thumb/d/dd/Gfp-wisconsin-madison-the-  
16                    },  
17                },  
18            ],  
19        }  
20    ],  
21    max_tokens=300,  
22 )  
23  
24 print(response.choices[0])
```


OpenAI Cookbook

New APIs

Introduction to Structured Outputs  Katia Gil Guzman Aug 6, 2024 COMPLETIONSFUNCTIONS	Introduction to GPT-4o and GPT-4o mini  Juston Forte Jul 18, 2024 COMPLETIONSVISIONWHISPER	Batch processing with the Batch API  Katia Gil Guzman Apr 24, 2024 BATCHCOMPLETIONS
Assistants API Overview (Python SDK)  Ilan Bigio Nov 10, 2023 ASSISTANTSFUNCTIONS	Processing and narrating a video with GPT's visual capabilities and the TTS API  Kai Chen Nov 6, 2023 COMPLETIONSPEECHVISION	Using GPT4 Vision with Function Calling  Shyamal Anadkat Apr 9, 2024 CHATVISION

Demo



Getting started with Azure OpenAI service



AZURE OPENAI SERVICE -KEY

- ## CONCEPTS
- **Prompt:** is a text input given to an AI model to elicit a specific response or output, guiding the model's generation process or decision-making.
 - **Completion:** is the output or response generated by an AI model based on a given prompt, representing the model's attempt to complete, answer, or extend the input it received.
 - **Tokens:** For text tokens Azure OpenAI processes text by breaking it down into tokens. Tokens can be words or just chunks of characters. For example, the word "hamburger" gets broken up into the tokens "ham", "bur" and "ger", while a short and common word like "pear" is a single token. Many tokens start with a whitespace, for example "hello" and "bye".
 - **Image tokens:** The token cost of an input image depends on two main factors: the size of the image and the detail setting (low or high) used for each image.
 - **Resources:** Azure OpenAI is a new product offering on Azure. You can get started with Azure OpenAI the same way as any other Azure product where you [create a resource](#), or instance of the service, in your Azure Subscription.
 - **Deployments:** Once you create an Azure OpenAI Resource, you must deploy a model before you can start making API calls and generating text.
 - **Endpoint:** Is a URL used to access OpenAI models on Azure, enabling applications to send requests and receive AI-generated responses via API calls.
 - **RAG:** RAG, or Retrieval-Augmented Generation, is a methodology in natural language processing that combines the retrieval of relevant documents or data with a generative model to enhance the generation of text.

AZURE OPENAI SERVICE –

1. Prerequisites

GETTING STARTED

1. An Azure subscription
2. Access to Azure OpenAI in the selected Azure subscription.
 - Currently, you must submit an application to access Azure OpenAI Service. To apply for access, complete [this form](#)

2. Create a resource

3. Configure network security

4. Deploy a model

1. Sign in to [Azure OpenAI Studio](#)
2. Choose the subscription and the Azure OpenAI resource to work with
3. Select **Create new deployment** and configure the needed fields

Resources:

[Create and deploy an Azure OpenAI Service resource](#)

The screenshot shows the 'Create Azure OpenAI' form in the 'Basics' tab. It includes sections for 'Project details' (Subscription, Resource group), 'Instance details' (Region, Name), and 'Pricing tier'. The 'Subscription' is set to 'OpenAI Test Subscription' and the 'Resource group' is 'test-resource-group'. The 'Region' is 'South Central US' and the 'Name' is 'azure-openai-test-001'. The 'Pricing tier' is 'Standard S0'. There is a 'Content review policy' section with links to learn more and apply for modified content filters.

The screenshot shows the 'Create Azure OpenAI' form in the 'Network' tab. It includes a section for 'Configure network security for your Azure AI services resource.' with a 'Type' dropdown set to 'All networks, including the internet, can access this resource.'

Deploy model

Set up a deployment to make API calls against a provided base model or a custom model. Finished deployments are available for use. Your deployment status will move to succeeded when the deployment is complete and ready for use.

Select a model ⓘ

gpt-35-turbo

Model version ⓘ

Auto-update to default

Auto-update to default

0301 (Default)

0613

Create

Cancel

AZURE OPENAI SERVICE –

You can start making calls to your deployed model via:

MAKING CALLS

- From The Language Studio
 - [Language Studio](#) is a set of UI-based tools that lets you explore, build, and integrate features from Azure AI Language into your applications.
 - [Getting started with Language Studio](#)
- Using SDKs
 - [Getting started with the Azure AI SDK](#)
 - [Azure OpenAI client library for .NET](#)
- Using REST API
 - [Azure OpenAI Service REST API](#)

